Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 5

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Патерни проектування»

Виконав
Студент групи ІА-31:
Козир Я. О.

Перевірив:
Мягкий М. Ю.

Київ 2025

Зміст

1. Мета: 3
2. Хід роботи 3
   1) Ознайомитись з короткими теоретичними відомостями. 3
   2) Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей. 3
   3) Реалізувати один з розглянутих шаблонів за обраною темою. 5
   4) Реалізувати не менше 3-х класів відповідно до обраної теми. 6
   5) Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону. 8
3. Контрольні запитання 8
4. Висновок 10

1. **Мета:**

Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

2. **Хід роботи**

Тема : Online radio station

1) Ознайомитись з короткими теоретичними відомостями.

Adapter: Адаптує інтерфейс одного об'єкта до іншого, дозволяючи несумісним компонентам працювати разом (наприклад, уніфікація інтерфейсів бібліотек принтерів). Спрощує інтеграцію без змін у коді.
Builder: Відокремлює процес створення об'єкта від його представлення, доречний для складних або багатоформних конструкцій (наприклад, відповіді веб-сервера). Забезпечує гнучкий контроль.
Command: Перетворює виклик методу в об'єкт, дозволяючи гнучкі системи команд із відміною, логуванням чи плануванням (наприклад, дії в інтерфейсі). Покращує модульність.
Chain of Responsibility: Передає запити по ланцюжку обробників, поки один не виконає (наприклад, контекстні меню). Зменшує зв'язки та спрощує зміни.
Prototype: Створює об'єкти клонуванням прототипу, корисний для складних об'єктів або динамічних змін (наприклад, редактор рівнів гри). Зменшує ієрархію спадкування.

2) Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```
public async Task<IActionResult> Add(Guid stationId)
{

    var users = await _userService.GetAllUsersAsync();
    var currentUser = users.FirstOrDefault();
    if (currentUser == null)
    {
        TempData["Error"] = "Помилка: Не вдалося визначити користувача.";
        return RedirectToAction("UserDashboard", "Home");
    }
    Guid userId = currentUser.UserId;

    try
    {
        var command = new AddFavoriteStationCommand(_favoriteService, userId, stationId);
        await command.ExecuteAsync();
        TempData["Message"] = "Станцію додано до улюблених!";
    }
    catch (Exception ex)
    {
        TempData["Error"] = $"Помилка додавання: {ex.Message}";
    }

    return RedirectToAction("UserDashboard", "Home");
}
```

Метод контролера (FavoritesController), що створює та виконує команду AddFavoriteStationCommand

```
public async Task AddFavoriteAsync(Guid userId, Guid stationId)
{
    bool alreadyFavorite = await _favoriteRepository.IsFavoriteAsync(userId, stationId);
    if (alreadyFavorite)
    {

        Console.WriteLine("Станція вже є в улюблених.");
        return;
    }

    // TODO: Додати перевірку, чи існують User та Station з такими ID

    var newFavorite = new FavoriteStation
    {
        FavoriteId = Guid.NewGuid(),
        UserId = userId,
        StationId = stationId
    };

    _favoriteRepository.AddEntity(newFavorite);
    await _favoriteRepository.SaveChangesAsync();
}
```

Метод сервісу (FavoriteService), що реалізує логіку додавання станції до улюблених

Реалізовано функціонал додавання радіостанцій до списку улюблених для залогіненого користувача. Ця дія інкапсульована за допомогою патерну "Команда". Контролер (FavoritesController) створює об'єкт команди (AddFavoriteStationCommand), який містить необхідні дані (ID користувача, ID станції) та посилання на сервіс (FavoriteService). Виконання команди делегується сервісу, який реалізує логіку перевірки та збереження даних у базу даних.

3) Реалізувати один з розглянутих шаблонів за обраною темою.

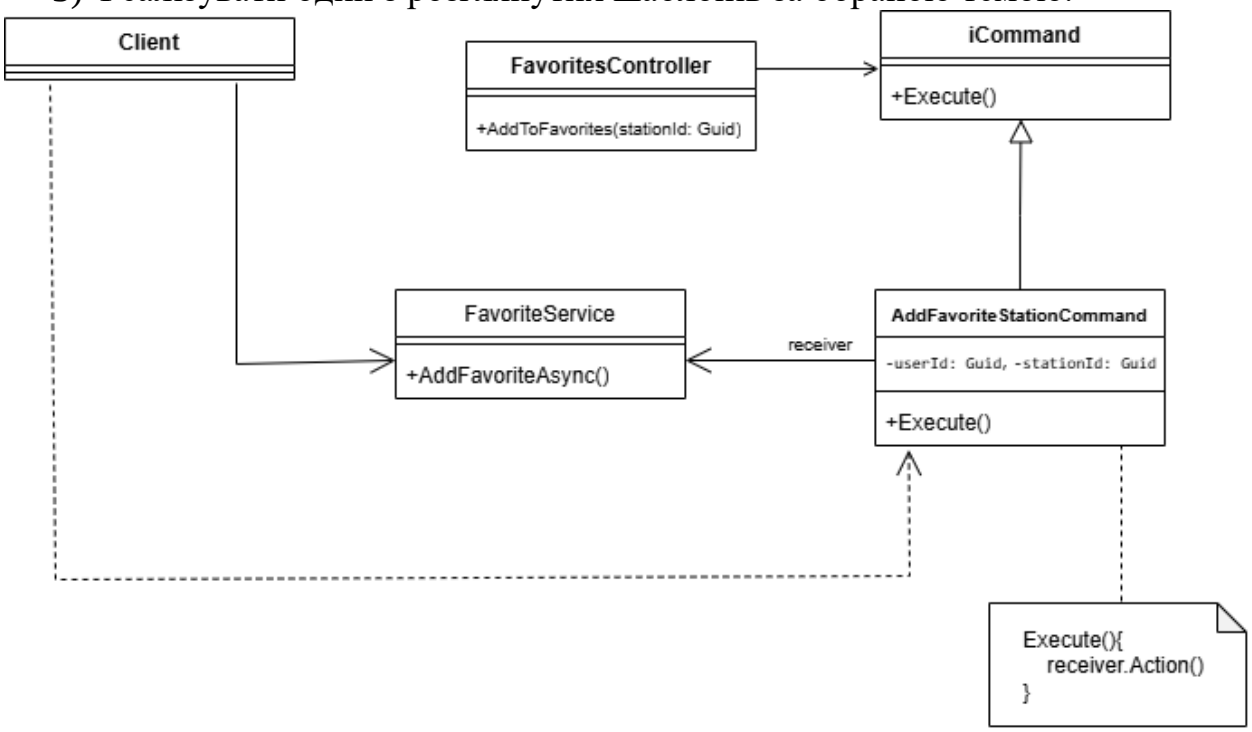


Рисунок 1 - Структура патерну Command

Ця діаграма ілюструє, як патерн "Команда" використовується для дії "**Додати станцію до улюблених**".

Client (Клієнт): Це ініціатор дії (наприклад, UI-елемент або інша частина програми). Він знає, яку дію потрібно виконати, і звертається до Викликача (FavoritesController). На діаграмі показано залежність (пунктир) від Викликача та потенційно від Команди (якщо клієнт створює її сам).

FavoritesController (Викликач / Invoker): Цей клас приймає запит від Клієнта.

- Метод +AddToFavorites(stationId: Guid): Створює конкретний об'єкт команди (AddFavoriteStationCommand) і викликає її метод Execute(). Він не знає деталей виконання команди, а лише ініціює її.
- Має асоціацію (проста стрілка) з інтерфейсом ICommand, показуючи, що він працює з командами через їхній загальний інтерфейс.

ICommand (Інтерфейс Команди): Визначає загальний інтерфейс для всіх команд. Метод +Execute(): Єдиний метод, який має бути реалізований усіма конкретними командами.

AddFavoriteStationCommand (Конкретна Команда): Інкапсулює всю інформацію, необхідну для виконання конкретної дії.

- Атрибути -userId: Guid, -stationId: Guid: Зберігає дані, потрібні для виконання (хто додає і яку станцію).
- Зв'язок receiver: Має посилання (асоціацію) на об'єкт Отримувача (FavoriteService), який безпосередньо виконає логіку.

- Метод +Execute(): Реалізує метод з інтерфейсу ICommand. Всередині він викликає відповідний метод Отримувача (наприклад, _favoriteService.AddFavoriteAsync(_userId, _stationId)), передаючи йому збережені дані.
- Спадкування: Реалізує інтерфейс ICommand (стрілка з порожнім трикутником).

FavoriteService (Отримувач / Receiver): Цей клас містить реальну бізнес-логіку для виконання дії. Метод +AddFavoriteAsync(): Виконує операцію додавання станції до улюблених (ймовірно, взаємодіючи з репозиторієм). Він нічого не знає про команду чи викликача; він просто робить свою роботу, коли його про це просять.

4) Реалізувати не менше 3-х класів відповідно до обраної теми.

```
using System.Threading.Tasks;

namespace OnlineRadioStation.Services
{
    1 reference
    public interface ICommand
    {
        2 references
        Task ExecuteAsync();
    }
}
```

Інтерфейс ICommand, що визначає контракт для всіх команд

```
namespace OnlineRadioStation.Services
{
    2 references
    public class AddFavoriteStationCommand : ICommand
    {
        2 references
        private readonly IFavoriteService _favoriteService;
        2 references
        private readonly Guid _userId;
        2 references
        private readonly Guid _stationId;

        1 reference
        public AddFavoriteStationCommand(IFavoriteService favoriteService, Guid userId, Guid stationId)
        {
            _favoriteService = favoriteService;
            _userId = userId;
            _stationId = stationId;
        }

        2 references
        public async Task ExecuteAsync()
        {
            await _favoriteService.AddFavoriteAsync(_userId, _stationId);
        }
```

Клас AddFavoriteStationCommand, що інкапсулює дію додавання до улюблених

```
using OnlineRadioStation.Domain;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace OnlineRadioStation.Services
{
    6 references
    public interface IFavoriteService
    {
        2 references
        Task AddFavoriteAsync(Guid userId, Guid stationId);

        1 reference
        Task RemoveFavoriteAsync(Guid userId, Guid stationId);

        1 reference
        Task<IEnumerable<RadioStationEntity>> GetUserFavoritesAsync(Guid userId);
    }
}
```

Інтерфейс IFavoriteService (Отримувач), що визначає операції з улюбленими станціями

Для реалізації патерну "Команда" та пов'язаного функціоналу було створено наступні класи та інтерфейси, що задовольняє вимогу створення щонайменше 3-х нових типів: інтерфейс ICommand, конкретна команда AddFavoriteStationCommand, інтерфейс сервісу-отримувача IFavoriteService та

його реалізація FavoriteService. Також було додано сутність FavoriteStation та відповідні репозиторії.

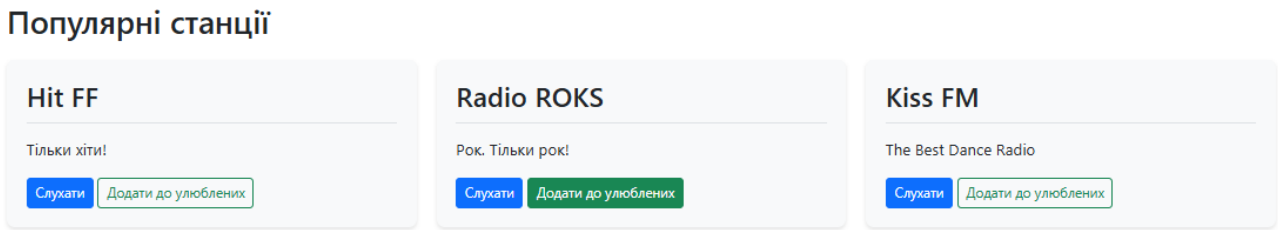


Рисунок 2 – Відображення станцій з можливістю додати до улюблених

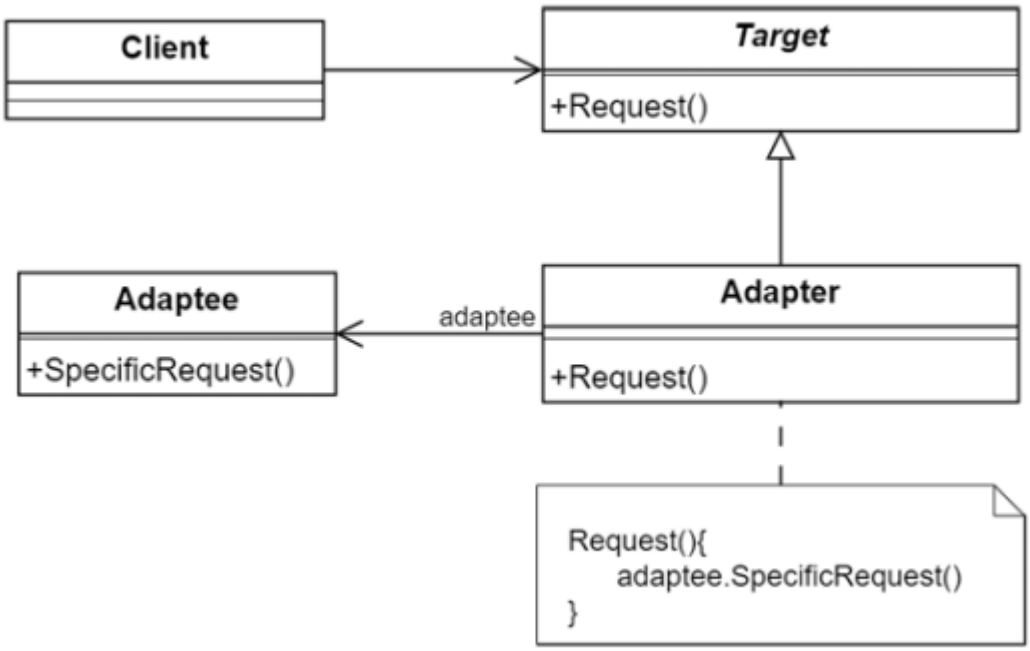
	FavoriteId	UserId	StationId
1	7CB455FF-9F24-4404-84A5-7DAA8FB9B523	CC39E904-D61D-40FB-B2EC-6F7E4411D560	CACE880F-3DE4-49A0-A49D-6AF6BD0716ED
2	ABCA4F35-5AF7-468A-B8DB-AE07F2015F08	CC39E904-D61D-40FB-B2EC-6F7E4411D560	1B8D7FDA-1AAE-492C-9CF1-E31CAC2BC264

Рисунок 3 – Відповідний запис станції та користувача, що її додав до улюблених в БД

5) Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

3. Контрольні запитання

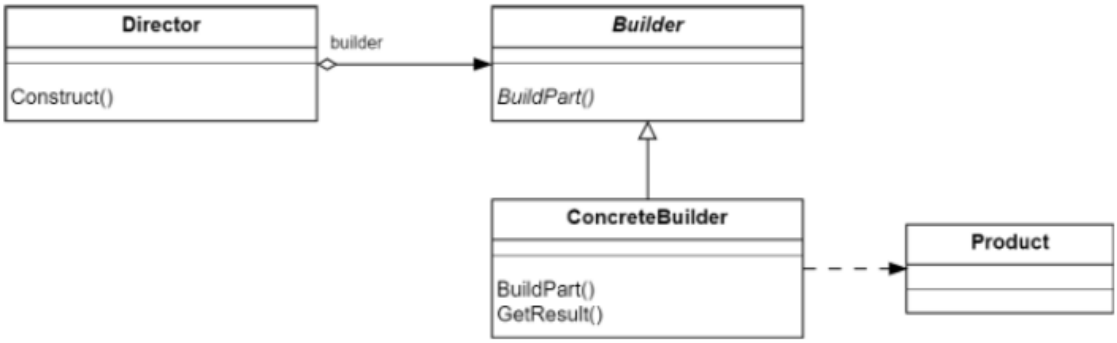
1) Яке призначення шаблону «Адаптер»? Адаптує інтерфейс одного об’єкта до іншого, дозволяючи несумісним компонентам працювати разом (наприклад, уніфікація бібліотек принтерів).
2) Нарисуйте структуру шаблону «Адаптер».



3) Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?

Client, Target, Adapter, Adaptee. Client працює з Target через адаптований інтерфейс, Adapter перенаправляє виклики до Adaptee.

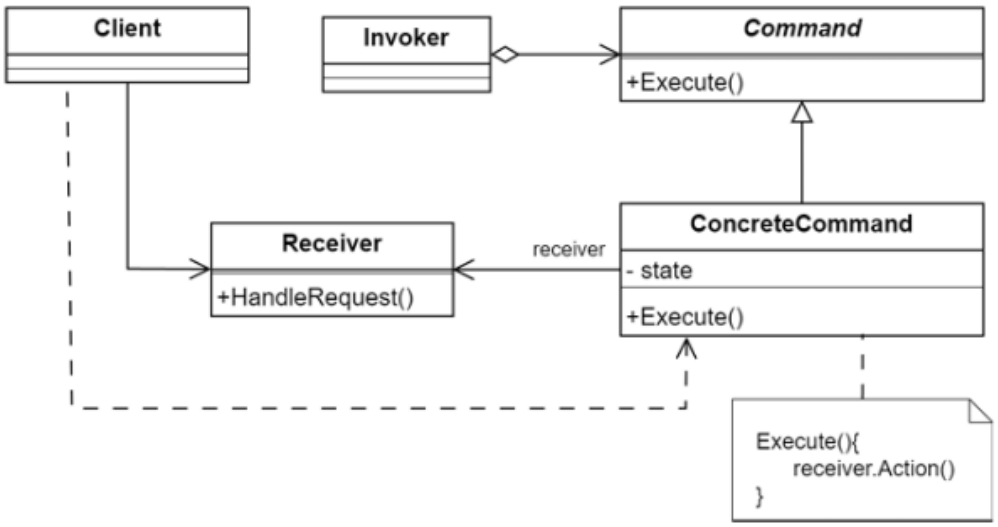
4) Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів? Об'єктний адаптер використовує композицію, класовим — успадкування.
5) Яке призначення шаблону «Будівельник»? Відокремлює процес створення об'єкта від його представлення, придатний для складних або багатоформних конструкцій.
6) Нарисуйте структуру шаблону «Будівельник».



7) Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?

Director, Builder, ConcreteBuilder, Product. Director керує будівництвом, Builder визначає інтерфейс, ConcreteBuilder реалізує його, Product — результат.

8) У яких випадках варто застосовувати шаблон «Будівельник»? При складному процесі створення або необхідності різних форм об'єкта.
9) Яке призначення шаблону «Команда»? Перетворює виклик методу в об'єкт для гнучкої системи команд із відміною, логуванням чи плануванням.
10) Нарисуйте структуру шаблону «Команда».



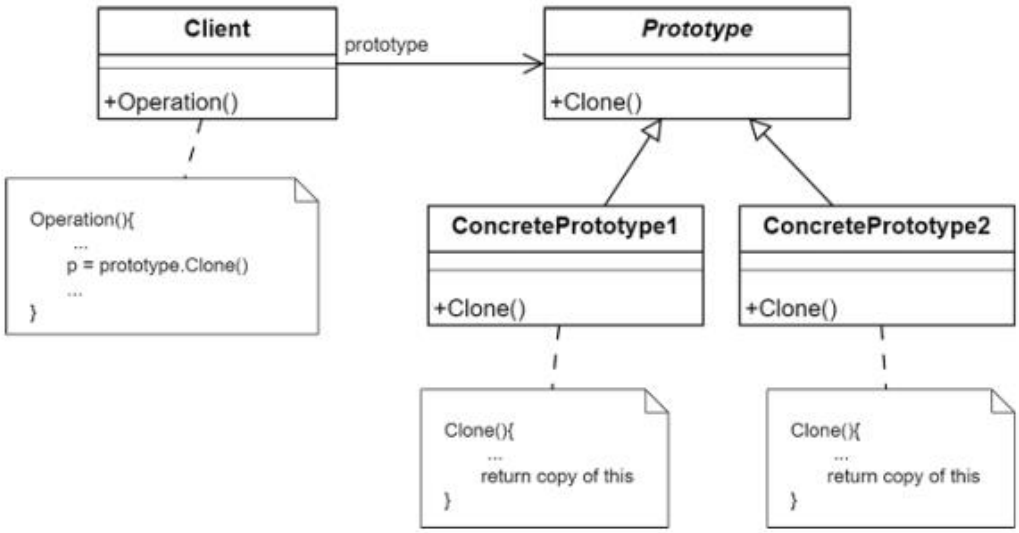
11) Які класи входять в шаблон «Команда», та яка між ними взаємодія?

Client, Invoker, Command, ConcreteCommand, Receiver. Client створює команду, Invoker виконує її, Receiver реалізує дію.

12) Розкажіть як працює шаблон «Команда». Команда інкапсулює запит як об’єкт, який передається Invoker до Receiver для виконання, підтримуючи додаткові функції (скасування, логування).

13) Яке призначення шаблону «Прототип»? Створює об’єкти клонуванням прототипу, зменшуючи ієрархію спадкування.

14) Нарисуйте структуру шаблону «Прототип».



15) Які класи входять в шаблон «Прототип», та яка між ними взаємодія?

Client, Prototype. Client клонувати об’єкт через метод Prototype.

16) Які можна привести приклади використання шаблону «Ланцюжок відповідальності»? Формування контекстного меню в UI, обробка запитів у ієрархії документів.

4. Висновок

У ході виконання лабораторної роботи було розглянуто базові шаблони проєктування, зокрема Adapter, Builder, Command, Chain of Responsibility та Prototype. Отримані знання дали можливість зрозуміти, яку роль відіграють патерни у побудові гнучких і масштабованих програмних систем. На прикладі веб-застосунку «Online Radio Station» було реалізовано шаблон Command для інкапсуляції дії "додавання радіостанції до улюблених" в окремий об'єкт (AddFavoriteStationCommand). Це дало змогу відокремити ініціатора запиту (контролер FavoritesController) від безпосереднього виконавця логіки (сервіс FavoriteService). Такий підхід робить систему більш гнучкою, оскільки сама дія

стає об'єктом, що в майбутньому дозволить легко додавати функціонал логування, скасування або постановки дій у чергу.